

04 — Operations

This document covers running and administering a MegaMOO server: starting it, configuring it, the permission model, persistence and backups, and the JSON API / MCP integration that lets external tools (including AI assistants) drive the live world.

- [Requirements](#)
- [Starting the server](#)
- [Command-line reference](#)
- [Creating a database](#)
- [Configuration](#)
- [The permission ladder](#)
- [Player administration](#)
- [Persistence and backups](#)
- [Logging](#)
- [Shutdown and restart](#)
- [The JSON API](#)
- [The MCP integration](#)

Requirements

- **Python 3.10+.** No `pip install` is needed for the core server — it runs on the standard library alone.
- **Optional:** `bcrypt` for stronger password hashing (a salted SHA-256 fallback is built in); `websockets` for browser clients; `mcp` for the AI integration bridge.

Starting the server

For everyday development use the launcher, which needs nothing but a database:

```
./mm mm.db
```

It enables the API, passes the shared token from `~/.megamoo/token` (generated on first run), turns on verb auto-reload, and names no ports — so a second database is started exactly the same way, with no arguments to keep distinct:

```
./mm ~/megamoo/mm.db
```

With no argument at all it picks the only `.db` in the repo root, and says so rather than guessing when there are several. Anything after the database is passed through to `megamoo.py` (`./mm mm.db --log-level DEBUG`).

The launcher is a thin wrapper over the real entry point, `megamoo.py` :

```
python3 megamoo.py mm.db
```

That loads the database `mm.db` and listens for Telnet on the **default port 6770** (`DEFAULT_PORT` in `moo/globals.py`) — or the next free port above it; see [Port selection and discovery](#). Connect with any MUD client or:

```
telnet localhost 6770
```

You'll see the splash screen and the prompt `Enter your username or NEW to create a new account: .`

Port note: the project README's quick-start uses `7777` as an *example* value, but the actual built-in default is `6770`. Pass `--port` (or a positional port) to pin one explicitly — which also turns off the search for a free port, so a conflict fails the boot instead of moving the game somewhere players aren't looking.

The argument parser supports both a LambdaMOO-style positional form and modern flags, and is smart about positional arguments — a second positional that is all digits is treated as a port, a hostname-like token as the bind host:

```
python3 megamoo.py mm.db                # db, default host/port
python3 megamoo.py mm.db 7777            # db on port 7777
python3 megamoo.py mm.db localhost 7777  # db, bind localhost, port 7777
python3 megamoo.py --input mm.db --port 8888 --host 127.0.0.1
```

On startup the server prints a banner with the version, loads the database, opens the listener(s), restores tickers, and starts its background tasks (checkpointing, the task queue, the ticker loop).

Status: the engine is at `0.10.0-beta20` and has **not yet been load- or play-tested**. Treat it as pre-release: suitable for development and single-developer worlds, not yet for an open multiplayer deployment. The version shown is `SERVER_VERSION` in `moo/globals.py`, the single source of truth for the build.

First login: the owner/wizard account (#100)

The shipped database includes **#100**, the default owner/wizard account — the first character in the game. It has full authority (`gm5` , plus the `WIZARD` and `PROGRAMMER` flags), so it can create players, build rooms, write verbs, and administer the server. `megamoo init` generates a password for this account and prints it once, when it creates the game:

```
Wizard login: Wizard / winter-timber-7382
Shown once, and unique to this world -- write it down.
```

Every world gets its own. There is no default password to look up, and none in the released package – a shipped hash is a shipped secret, and this one was recoverable from the wheel in under a second.

If you have lost it, there is no recovery path from outside the game; create a fresh world, or set the property directly against the database with the server stopped.

Two things to do on first login, before anything else:

1. **Change the password.** Use `password` (interactive) or `setpass <newpassword>` from the OOC lobby. **Do this immediately** in any environment reachable by others — it is the first item on the [security checklist](#).
2. **Make it yours — rename #100.** This account is meant to *become* the owner's own character, not stay a generic "Wizard." Rename it with `@rname #100 = <YourName>` , which moves the login name with it — you log in as the new name from then on, and `Wizard` stops working.

Use `@rname` , not `@name` , for an account. `@name` renames the object and leaves the login where it was, so you would end up called by your own name while still typing `Wizard` at the prompt. `@name` remains the right verb for everything that is not an account (see naming objects).

From here you grant other staff their tiers with `@auth` (see [the permission ladder](#)).

Keep at least one account at `gm5` . If you rename and re-secure #100 it remains your god account; only create additional `gm5` accounts deliberately.

Command-line reference

Flag	Default	Purpose
database (positional) / --input, -i	— (required)	Database file to load.
new_database (positional) / --output, -o	—	Create a new database from the template (see below).
port (positional) / --port, -p	first free from 6770 up	TCP port for the Telnet listener. Naming one pins it: a conflict then fails the boot.
--host	0.0.0.0	Bind address.
--config, -c	—	Path to a ServerConfig JSON file (see Configuration).
--api	off	Enable the JSON API server (first free port from 7778 up).
--api-port	auto	Pin the API to this exact port; fails if it is in use.
--api-token	—	Shared secret API clients must present.
--web	off	Enable the browser client: serves the client and accepts WebSocket players (first free port from 8888 up).
--web-port	auto	Pin the browser client to this exact port; fails if it is in use.
--web-host	follows --host	Bind the browser client somewhere the game listener isn't. Set 127.0.0.1 when a reverse proxy fronts the client, so the plain HTTP port isn't reachable from outside the box.
--web-origins	same-origin only	Comma-separated origins allowed to open a WebSocket in addition to the one the client was served from (e.g. https://play.example.com). Needed only when the client is served from somewhere other than this server. * accepts any origin.
--web-tls	off	Serve the browser client over HTTPS, using --tls-cert / --tls-key. A flag on the web port rather than a second one, because the client picks ws:// or

Flag	Default	Purpose
		<code>wss://</code> from the page it was served over.
<code>--tls-port</code>	—	Serve an additional TLS listener on this exact port. The plain port keeps working — telnet cannot speak TLS — so this is a second door to the same world. Requires <code>--tls-cert</code> and <code>--tls-key</code> .
<code>--tls-cert</code>	—	PEM certificate for <code>--tls-port</code> , including any intermediates.
<code>--tls-key</code>	—	PEM private key for <code>--tls-port</code> .
<code>--log-level</code>	INFO	DEBUG, INFO, WARNING, or ERROR.
<code>--dev</code>	off	Development mode: pick the lone <code>.db</code> here, reuse the shared API token, publish a discovery file so tooling can find the world, and hot-reload verbs from disk. Turns on <code>--api</code> and <code>--web</code> as well.
<code>--version</code> , <code>-v</code>	—	Print the version and exit.
<code>-h</code> , <code>--help</code>	—	Full help.

Creating a database

Give the server a **template** plus an **output path** and it will build a fresh database from the template and exit, telling you how to launch it:

```
python3 megamoo.py template.db newworld.db
# ...creates newworld.db, then:
python3 megamoo.py newworld.db
```

Pointing the server at a database file that doesn't exist (with no output path) is an error — it won't silently create an empty world; you start from a template.

The shipped database. MegaMOO ships with a default database containing the core object hierarchy — every object through **#44** — and **#100**, the owner/wizard account and the first character in the game. That gives you the base prototypes (rooms, exits, containers, furniture) and a usable login from the first launch; build your world on top of it.

Configuration

Two distinct files are easy to confuse:

- **`config.json` in the repo root is *not* the server config.** It configures an unrelated document/vector-search tool (embeddings model, chunk size). The server does not read it.
- **The server's `--config` expects a `ServerConfig` JSON** with `network`, `database`, `protocol`, `api`, and `dev` sections, matching the dataclasses in `moo/config.py`. Anything omitted falls back to the documented default. `ServerConfig` also carries top-level scalars — `server_name`, `motd`, `login_welcome`, `display_screen`, `max_command_length`, `enable_color`, `debug_mode`, `log_level`.

A representative config (all values shown are the defaults):

```

{
  "network": {
    "host": "0.0.0.0",
    "port": 6770,
    "auto_port": true,
    "port_scan_limit": 50,
    "max_connections": 100,
    "connection_timeout": 3600,
    "max_players": 0,
    "tls_port": 0,
    "websocket_enabled": false,
    "websocket_port": 8888
  },
  "database": {
    "path": "game.db",
    "checkpoint_interval": 3600,
    "max_checkpoints": 10,
    "transaction_log_enabled": true,
    "auto_save_interval": 300,
    "compress_checkpoints": true,
    "backup_on_start": true,
    "max_object_cache": 1000
  },
  "protocol": {
    "...": "MXP / GMCP / MSDP / MSSP toggles"
  },
  "api": {
    "enabled": false,
    "port": 7778,
    "auto_port": true,
    "port_scan_limit": 50,
    "info_path": "",
    "host": "127.0.0.1",
    "auth_token": "",
    "testbot_objnum": 0
  },
  "dev": {
    "autoreload_verbs": true,
    "autoreload_interval": 2.0
  }
}

```

Notes on the fields that matter most in practice:

- **network.max_players: 0** means unlimited; **max_connections** bounds raw sockets.
- **network.auto_port** lets a busy **port** move the listener up to the next free one instead of failing the boot, which is what makes a second database a one-word command. Set it **false** (or name a port on the command line) when the server must be reachable at a fixed address.

- `database.auto_save_interval` (5 min) is the in-memory-to-disk save; `checkpoint_interval` (1 hr) writes pruned recovery snapshots; the two are independent. `backup_on_start` takes a full copy before the server begins writing.
- `api.host` defaults to `127.0.0.1` — keep it that way unless you really mean to expose the API, and always set a strong `auth_token`. `testbot_objnum` pins the API's driver character (see [MCP](#)).
- `dev.autoreload_verbs` is **true by default**. A background watcher polls `moo verbs/` every `dev.autoreload_interval` seconds and hot-loads any verb file whose mtime changed, so external edits go live without `@reload` (see the auto-reload watcher). Set it to `false` if you want disk edits to land only when explicitly pulled.

The config validates on load: SSL requires a cert/key, an enabled API requires a valid port, etc.

The permission ladder

Authorization is a five-level ladder stored in each character's `auth` property and managed in-game. Two object flags are kept in sync automatically: granting `gm3` sets `PROGRAMMER`, granting `gm4` sets `WIZARD`.

Level	Name	Can, roughly
gm1	AssistantGM	Inspect (<code>+show</code>), teleport, basic helpers.
gm2	Builder	Full world-building: rooms, exits, objects, descriptions, messages.
gm3	Coder	Programming: create from any parent, add/remove properties and verbs, reparent, <code>eval</code> , inspect internals.
gm4	Admin	Player administration (<code>@delplayer</code>).
gm5	God	Server control (<code>@shutdown</code> / <code>@restart</code>), renumbering, number reservation, granting auth.

Grant and inspect with `@auth` (gm5 only):

```
@auth bob = add gm2
@auth bob = remove gm3
@auth bob = list
```

Per-verb authorization is independent of who *owns* a verb: `@verbauth` and the `auth=N` option on `@adverb` set the minimum level required to invoke a specific verb (see Building Worlds).

Player administration

Command	Level	What it does
<code>@auth <player> = add\ remove <level> / = list</code>	gm5	Manage a player's auth levels (syncs PROGRAMMER / WIZARD).
<code>@delplayer <object#></code>	gm4	Permanently delete an account: moves its IC characters and inventories to storage (#8), clears local properties, returns the account to the player pool (#2). Requires typing <code>YES</code> to confirm.

Persistence and backups

State lives in **SQLite** with WAL journaling. In normal operation you do not issue saves — property writes are persisted through automatically (see Architecture). For operations:

- **Auto-save** flushes memory to disk every `auto_save_interval` seconds (default 300).
- **Checkpoints** are written every `checkpoint_interval` seconds (default 3600) into a directory named after the database file and placed beside it — `mm.db` → `mm_checkpoints/`, holding `checkpoint_<timestamp>.sqlite` snapshots — with `max_checkpoints` retained and optional gzip.
- **backup_on_start** copies the database before the server begins writing.
- The WAL files (`*.db-wal`, `*.db-shm`) alongside the database are normal SQLite artifacts; don't delete them out from under a running server.

To take a manual backup, the safest path is to `@shutdown` (which saves and unpuppets everyone) and copy the `.db` file. A live file copy can be torn if the server is mid-write — acceptable for a scratch snapshot, not for an authoritative backup.

Logging

The server logs to `megamoo.log` in the working directory via a rotating file handler (10 MB per file, 5 backups) and also to the console. The level defaults to `INFO`; raise it with `--log-level DEBUG`

when diagnosing. Because the log is a plain file, it can be tailed even when the server is down — the MCP bridge exposes exactly that (`tail_log`).

Shutdown and restart

Both are gm5 verbs and both save the database, broadcast an optional message, and unpuppet every connected character before disconnecting:

```
@shutdown Server going down for maintenance.
@restart Applying a hotfix, back in a moment.
```

`@restart` re-executes the server process after a clean save, so it picks up changes to **engine Python under moo/** — builtins, verb types, the parser, the namespace builder. That is the one category that is not hot.

It re-runs the same command line, so every launch flag survives — `--dev` , `--web` , `--port` , `--log-level` , the database — and so does the working directory. Two flags are handled specially, and deliberately not the same way:

	Behaviour on restart	Opt out / in
<code>--api</code>	Forced on , however the server was launched, so tooling reconnects without anyone remembering a flag.	<code>@restart/noapi</code> (or the bare word <code>noapi</code>)
<code>--web</code>	Left as it was . Only ever added, never stripped.	<code>@restart/web</code> adds it

The asymmetry is the point: the API is one loopback socket, while the browser client is reachable by anything that can reach the host, and without `--web-origins` that is an exposure rather than a convenience. A restart must not switch it on for a server that never asked.

`--dev` implies both, so on a development server neither switch changes anything.

You almost never need it for game content: verbs written in-game with `@program` are live immediately, and verbs edited on disk are hot-loaded by the auto-reload watcher within a couple of seconds (or on demand with `@reload`). See Hot coding.

The JSON API

When started with `--api` , the server runs a newline-delimited JSON-over-TCP API (`moo/api.py`), **off by default and localhost-only** by default. It is the control plane for external tooling — editors, scripts, and the MCP bridge. Enable it with a token:

```
python3 megamoo.py mm.db --api --api-token <token>
```

The API authenticates with the shared `auth_token` and drives a dedicated character (`testbot_objnum` , auto-provisioned if unset — see below). Keep `api.host` at `127.0.0.1` unless you have a specific reason to expose it, and always set a strong token.

Port selection and discovery

You do not assign ports by hand — neither the game's nor the API's. Each starts at its configured port (6770 and 7778) and, if that is taken, walks upward to the first free one, so every database is launched with the same command:

```
./mm mm.db           # game on 6770, API on 7778
./mm scratch.db      # game on 6771, API on 7779 — nothing to coordinate
```

Each move is logged (Port 6770 in use; auto-selected 6771 , API port 7778 in use; auto-selected 7779), and the pair that won is written to a **discovery file**:

```
{ "host": "127.0.0.1", "port": 7779, "pid": 38506,
  "database": "/Users/you/megamoo/mm.db", "auth_required": true,
  "game_port": 6771 }
```

`port` is the API's; `game_port` is the telnet port a MUD client connects to. Local tooling reads this instead of hard-coding a port — the MCP bridge does so automatically (see below). The file is removed on clean shutdown, and one left behind by a crash is ignored because its `pid` no longer names a live process. `server_status` also reports the live `api_port` and `database` .

Where it is written depends on how the server was started. `megamoo.py` puts it beside the database (`mm.db` → `mm.db.api.json`); `./mm` redirects it to `~/.megamoo/run/` so that tooling can see servers started from *other* checkouts too, under a filename built from the database's full path. The bridge searches both places.

Settings, per section: `network.auto_port` / `network.port_scan_limit` for the game listener and `api.auto_port` / `api.port_scan_limit` for the API (default 50 ports each); `api.info_path` overrides the discovery file's location, and `"-"` disables it.

Naming a port opts out of the search on that listener — `--port 6777` or `--api-port 7900` mean you want *that* port, so a conflict is a startup error (non-zero exit) rather than a silent move elsewhere.

The MCP integration

MegaMOO ships an [MCP](#) bridge (`tools/megamoo_mcp.py`) so an MCP client — such as Claude Code — can drive the running game directly: run commands as a bot character, inspect and edit live objects and verbs, search, and read logs, without touching the database by hand.

```
graph LR
    CC[Claude Code] -- stdio --> MB["tools/megamoo_mcp.py  
(MCP bridge)"]
    MB -- "TCP/JSON" --> AS["ApiServer (port 7778)  
inside running MegaMOO"]
    MB -- reads --> ML["megamoo.log (works even when server is down)"]
```

The bridge is a standalone FastMCP process; the game server gains no new dependencies.

One-time setup

```
pip install mcp
./mm mm.db
claude mcp add megamoo -- python3 ~/megamoo/tools/megamoo_mcp.py
```

No token in the registration: the bridge falls back to `~/.megamoo/token` , the same file `./mm` launches every database with, so rotating it needs no config edit. Pass `-e MEGAMOO_API_TOKEN=<token>` if you'd rather set it explicitly — the environment wins.

Tools

Game tools (require the server running):

Tool	Description
<code>run_command</code>	Execute a game command as the bot character; returns its output (color stripped).
<code>disconnect_testbot</code>	Cleanly unpuppet the bot via the normal <code>on_unpuppet</code> path.
<code>get_object</code>	Full object info: name, parent, flags, properties, verbs.
<code>get_location</code> / <code>list_contents</code>	An object's location / a room or container's contents.
<code>list_verbs</code> / <code>get_verb</code> / <code>set_verb</code> / <code>delete_verb</code>	Read and write verb source live.
<code>list_properties</code> / <code>get_property</code> / <code>set_property</code>	Read and write properties.
<code>eval_code</code>	Evaluate arbitrary Python in the verb context.
<code>search_verbs</code> / <code>search_objects</code>	Full-text search across verb source / objects by name or description.

Disk tools (work even when the server is down):

Tool	Description
<code>tail_log</code>	Last N lines of <code>megamoo.log</code> , with an optional regex filter.
<code>server_status</code>	API reachability; if up, uptime, player count, and the database/port in use.
<code>list_servers</code>	Every running server: database, telnet port, API port.
<code>use_database</code>	Point the bridge at one of them for the rest of the session.

Working with several databases

Nothing is pinned to one database. With more than one server up, ask the assistant which are running and which to work on:

you: what MegaMOO servers are running? **assistant:** (`list_servers`) two — `mm.db` (game 6770, API 7778) and `scratch.db` (game 6771, API 7779). **you:** switch to scratch **assistant:** (`use_database("scratch")`) now on `/Users/you/megamoo/scratch.db`.

`use_database` accepts `mm`, `mm.db`, or a full path, takes effect on the next call, and needs no restart of the client or the bridge. An empty string returns it to auto-discovery.

Finding the server

The bridge resolves the API address on every (re)connect, so it follows a server that restarted onto a different port without restarting the MCP process:

1. `MEGAMOO_API_PORT` — an explicit pin, skipping discovery.
2. A database selected this session with `use_database`.
3. `MEGAMOO_API_INFO` — a named discovery file.
4. `MEGAMOO_DB` — that database's discovery file (`<db>.api.json`).
5. The only live discovery file in `~/.megamoo/run/` or the repo root — the everyday case: one server up, on whatever port it selected.
6. Otherwise port 7778.

With several servers advertising and no hint, the bridge refuses to guess and lists what it found.

`MEGAMOO_DB` is still there for a permanent default, but `use_database` is the everyday answer — it costs a sentence instead of a config edit.

Notes

- The bot character is **auto-created on first run_command** if no character of the configured name exists under `#5 ICharacter`. To pin a specific character, set `api.testbot_objnum` before starting the server.
- The integration test suite (`python3 -m pytest tests/`) boots its own server on separate ports against a scratch copy of the database, so it can run alongside a dev server. Two caveats: the test subprocess appends to the shared `megamoo.log`, and the database snapshot is a plain file copy (could tear under heavy live writes). When in doubt, stop the server or run only the unit tests: `python3 -m pytest tests/ --ignore=tests/test_integration_api.py`.

Security checklist

Before exposing a server beyond your own machine:

1. **Change the wizard password** from the one `megamoo init` generated, or at least confirm you still have it (see [First login](#)).
2. **Keep the JSON API local.** Leave `api.host` at `127.0.0.1` and set a strong `api.auth_token`; only widen it deliberately.
3. **Enable TLS** (`tls_port` with `tls_cert / tls_key`, or the `--tls-port` flags) if the server is reachable over the internet. It is an additional listener: the plain port keeps working, because `telnet` cannot speak TLS. Passwords cross the plain port in cleartext.
4. **Decide where TLS terminates for the browser client.** Either the engine holds the certificate (`--web-tls`), or a reverse proxy does and the engine serves plain HTTP behind it. The proxy is usually the better answer: the engine builds its SSL context once at boot, so a renewed certificate only takes effect on a restart — and a restart disconnects every player. If you use a proxy, add `--web-host 127.0.0.1` so the plain port is not independently reachable, and don't rely on a firewall rule to hide it.
5. **Check the origins if you serve the client from a second domain.** The default is same-origin only, which is what a browser loading the client from this server needs. `--web-origins` adds origins for a separate front end; `*` accepts any origin and should not survive contact with a public deployment.
6. **Turn off debug mode** in production so internal state isn't exposed.
7. **Right-size** `max_connections` for your hardware, and review the per-IP connection rate limit.
8. **Keep** `backup_on_start` **on** (default) and set up **external backups** of the database.
9. **Grant the minimum auth tier.** Give staff the lowest `gm` level that lets them do their job (see [the permission ladder](#)), and keep the set of `gm5` accounts small.

Next: Engine Systems — hooks, the ticker, and the effects system; then The Prototype Library.